

دليل مشروع تصنيف الأدوات المكتبية من الصفر

Office Tools Classification From Scratch Using CNN

القسم الأول: مقدمة المشروع وفلسفة التصميم (Project Overview & Architecture Justification)

١. فكرة المشروع والهدف منه:

يُعد هذا المشروع نظام رؤية حاسوبية (Computer Vision) متكامل، تم بناؤه من الصفر (From Scratch) باستخدام مكتبة PyTorch. يهدف النظام إلى حل مشكلة تصنيف الصور (Image Classification) لثلاث فئات من الأدوات المكتبية الأساسية:

- الأقلام (Pens)
- المقصات (Scissors)
- الدبابسات (Staplers)

٢. التأصيل العلمي للمشروع:

- محاكاة المخ البشري (Biological Inspiration):** هذا النموذج طريقة عمل الخلايا العصبية البيولوجية؛ حيث تدخل الصور إلى الشبكة كإشارات (Inputs)، وتمر عبر طبقات متصلة ببعضها (Interconnected) تقوم الخلية العصبية الاصطناعية بجمع هذه المدخلات وضربها في أوزان محددة، وإذا تجاوزت النتيجة (Network) حداً معيناً (Threshold)، تقوم بتمرير الإشارة للطبقة التالية.
- الفرق بين التعلم العميق والتعلم الآلي التقليدي:** في التعلم الآلي التقليدي (Machine Learning)، يتم استخلاص الميزات من البيانات يدوياً (Human Feature Engineering). أما في هذا المشروع، نعتمد على التعلم العميق (Deep Learning) حيث يتعلم النموذج ويستخلص الميزات والأنماط تلقائياً من البيانات عبر الفلاتر الموجودة في الطبقات المخفية (Hidden Layers).

٣. لماذا تم اختيار معمارية الشبكات التلافيفية (CNN) تحديداً؟

تم اختيار شبكة (CNN) Convolutional Neural Network لأسباب علمية تضمن تفوقها على الشبكات العصبية الاصطناعية التقليدية (MLPs):

- الحفاظ على البنية المكانية للصور (Spatial Structure Retention):** للشبكات التقليدية تتطلب تسطيح الصورة بالكامل إلى متجه أحادي البعد قبل المعالجة، مما يؤدي إلى فقدان العلاقات المكانية بين وحدات البكسل (Pixels). أما الـ CNN فتتعامل مع الصورة كبنية ثنائية الأبعاد (2D Structure)، مما يحافظ على تماسك الملامح المكانية كالحواف والأشكال الهندسية.
- خاصية مشاركة الأوزان (Weight Sharing):** في الـ CNN، يتم تمرير فلاتر صغيرة (Kernels) تتحرك عبر الصورة. الفلتر الذي يكتشف حافة معينة في أعلى الصورة يمكنه اكتشافها في أسفل الصورة باستخدام نفس الأوزان. هذا يقلل بشكل كبير من عدد المعلمات (Parameters) التي يحتاج النموذج لتعلمها، مما يسرع العمليات الحسابية ويحمي النموذج من مشكلة التجهيز الزائد (Overfitting).

3. الفصل بين استخراج الميزات والتصنيف (Feature Extraction vs. Classification) تقسم الـ

المعالجة إلى مرحلتين CNN:

- **مرحلة استخراج الميزات**: تعتمد على طبقات Convolution و Pooling لاستخلاص أدق التفاصيل من الصور تلقائياً.
- **مرحلة التصنيف**: تعتمد على Fully Connected Layers التي تستقبل هذه الميزات الجاهزة وتتخذ قرار التصنيف النهائي.

القسم الثاني: هندسة البيانات ومعالجتها (Data Preprocessing & Augmentation)

```
from google.colab import drive
drive.mount('/content/drive')
```

الشرح: يتم استخدام هذين السطرين لربط بيئة عمل Google Colab بحساب Google Drive، مما يتيح للنموذج الوصول إلى مجلدات الصور لقراءتها ومعالجتها.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

الشرح: استيراد المكتبات الأساسية:

- **torch** : المكتبة الأساسية للتعلم العميق.
- **nn** وحدة الشبكات العصبية، وتحتوي على الطبقات الجاهزة كطبقات الالتفاف (Conv) والطبقات الخطية (Linear).
- **F** : وحدة الدوال الوظيفية (Functional API) التي تحتوي على دوال التنشيط مثل ReLU.
- **optim** : وحدة تحتوي على خوارزميات تحسين الأوزان (Optimizers).
- **torchvision** : مكتبة متخصصة للتعامل مع الصور وتجهيزها.

```
data_transforms = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

الشرح الأكاديمي لمعالجة البيانات:

- `transforms.Compose` : دالة تقوم بتجميع عدة عمليات معالجة ليتم تطبيقها بتسلسل منطقي.
- `transforms.Resize((32, 32))` : توحيد أبعاد جميع الصور لتصبح 32×32 بكسل. هذا يقلل من التعقيد الحسابي ويجعل النموذج قابلاً للتوسع (Scalable).
- `ColorJitter` `RandomRotation` `RandomHorizontalFlip` : هذه العمليات تُعرف بـ (Data Augmentation) أو زيادة البيانات. تقوم بتعديل الصور عشوائياً (كالقلب والدوران وتغيير الإضاءة) لتوليد بيانات (Augmentation) تدريب جديدة، مما يمنع النموذج من حفظ البيانات (Overfitting) ويجعله قادراً على التعرف على الأدوات في ظروف إضاءة وزوايا مختلفة.
- `transforms.ToTensor()` : تحويل الصورة من تنسيق بكسلات رقمية إلى مصفوفة رياضية (Tensor) بأبعاد [Channels, Height, Width]. 3. كما تقوم الدالة بقسمة (RGB) وبما أن الصور ملونة، فإن عدد القنوات هو 3. القيم على 255 لتجيمها بين [1, 0].
- `transforms.Normalize` : عملية التطبيع الرياضي. تقوم بطرح 0.5 وقسمة الناتج على 0.5 لتحويل نطاق قيم البكسلات لتصبح بين [-1, 1]، مما يحسن من استقرار العمليات الحسابية ويسرع من وصول الشبكة لأفضل أوزان (Convergence).

```
train_path = '/content/drive/MyDrive/Office_Tools_Project/train'
test_path = '/content/drive/MyDrive/Office_Tools_Project/test'

train_dataset = datasets.ImageFolder(root=train_path, transform=data_transforms)
test_dataset = datasets.ImageFolder(root=test_path, transform=data_transforms)
```

الشرح: استخدام دالة `ImageFolder` لإنشاء هيكل البيانات. هذه الدالة تقرأ المسار وتستنتج تلقائياً أن كل مجلد فرعي يمثل فئة (Class)، ثم تطبق عمليات المعالجة (`data_transforms`) على الصور.

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

الشرح: يقوم `DataLoader` بتقسيم البيانات إلى حزم صغيرة (Batches).

- `batch_size=32` : النموذج سيقوم بمعالجة 32 صورة في وقت واحد، ثم يحسب الخطأ ويحدث الأوزان.

- `shuffle=True` في مرحلة التدريب، يتم خلط الصور في كل دورة (Epoch) لضمان عدم حفظ النموذج لترتيب إدخال البيانات. في مرحلة الاختبار (`test_loader`)، لا نحتاج للخلط فنجعلها `False`.

القسم الثالث: المعمارية الهندسية للشبكة (CNN Architecture)

```
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
```

الشرح: تعريف الفئة `CNNModel` التي ترث خصائص `nn.Module` لتمكين استخدام أدوات PyTorch.

```
self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
```

شرح طبقات الالتفاف (Convolution Layers - Feature Extraction):

- `conv1`: تستقبل 3 قنوات (RGB) وتطبق 16 فلترًا. حجم الفلتر `kernel_size=3` (مصفوفة 3×3). تم تعيين الحشوة `padding=1` للحفاظ على أبعاد الصورة ومنع انكماشها، وكذلك لحماية التفاصيل (Same Padding) الموجودة على حواف الصورة.
- `conv2`: تستقبل الـ 16 قناة الناتجة عن الطبقة الأولى وتخرج 32 قناة بفلتر 3×3 وحشوة 1.
- `conv3`: تستقبل الـ 32 قناة وتخرج 64 قناة بنفس الإعدادات السابقة.

```
self.pool = nn.MaxPool2d(2, 2)
```

الشرح: تعريف طبقة التجميع (Max Pooling).

- **الوظيفة:** تقوم بعملية (Down-sampling) لتقليل أبعاد مصفوفة الميزات وتقليل القوة الحسابية، وتعمل أيضاً على إخماد الضوضاء (Noise Suppressant).
- القيم (2, 2) تعني استخدام فلتر بحجم 2×2 ومقدار خطوة (Stride) يساوي 2. هذه الطبقة تستخرج أعلى قيمة في كل مربع 2×2، مما يؤدي إلى تقليص حجم الصورة للنصف تماماً طولاً وعرضاً.

```
self.fc1 = nn.Linear(64 * 4 * 4, 128)
self.fc2 = nn.Linear(128, 3)
```

شرح الطبقات كاملة الاتصال (Fully Connected Layers - Classification):

- `fc1`: طبقة مخفية تستقبل المخرجات من الطبقة السابقة بعد عملية التسطيح، والتي يبلغ طولها (64 قناة × 4 طول × 4 عرض = 1024 عنصر)، وتتصل بـ 128 خلية عصبية (Neurons).

- طبقة `fc2` المخرجات النهائية، تستقبل الـ 128 خلية وتخرج 3 أرقام فقط (Logits) تمثل الفئات الثلاث للمشروع.

مسار البيانات وحساب الأبعاد (Forward Pass):

```
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = self.pool(F.relu(self.conv3(x)))
```

العمليات الحسابية المتسلسلة:

1. **البلوك الأول:** تدخل الصورة بأبعاد `[3, 32, 32]`. الـ `conv1` تحافظ على الأبعاد مع زيادة القنوات لتصبح `[16, 32, 32]`. تُطبق دالة التنشيط ReLU لإضافة خاصية اللاخطية (Non-linearity) وإلغاء القيم السالبة. ثم تقوم طبقة `pool` بتقليص الأبعاد للنصف لتصبح `[16, 16, 16]`.
2. **البلوك الثاني:** تقوم `conv2` بزيادة القنوات إلى 32، وتقوم طبقة `pool` بتقليص الأبعاد مرة أخرى لتصبح `[32, 8, 8]`.
3. **البلوك الثالث:** تقوم `conv3` بزيادة القنوات إلى 64، وتقوم طبقة `pool` بتقليص الأبعاد للمرة الأخيرة لتصبح `[64, 4, 4]`.

```
x = x.view(-1, 64 * 4 * 4)
```

الشرح: عملية التسطيح (Flattening)؛ وهي تحويل المصفوفة ثلاثية الأبعاد الناتجة عن آخر عملية Pooling إلى متجه خطي أحادي البعد جاهز ليدخل إلى طبقات الـ Classification. إجمالي العناوين هو $64 \times 4 \times 4 = 1024$ المعامل 1- يوجه بايثون لاحتساب حجم الحزمة (Batch) تلقائياً.

```
x = F.relu(self.fc1(x))
x = self.fc2(x)
return x
```

الشرح: تمرير المتجه الخطي المكون من 1024 عنصراً عبر الطبقة المخفية `fc1`، ثم تطبيق دالة ReLU، وأخيراً تمرير الناتج إلى طبقة الإخراج `fc2` لإنتاج التوقعات الثلاثة.

القسم الرابع: مرحلة التدريب (Training Loop & Loss Function)

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

الشرح:

- `nn.CrossEntropyLoss` : دالة التكلفة (Loss Function) الأنسب لمشاكل التصنيف متعدد الفئات. تحتوي هذه الدالة داخلياً على دال `Softmax` التي تحول المخرجات الخام إلى احتمالات مجموعها يساوي 1، وتقوم بحساب مقدار الخطأ بين التوقعات والواقع.
- `optim.Adam` خوارزمية التحسين المسؤولة عن تحديث الأوزان لتقليل الخطأ باستخدام تقنية الانحدار التدريجي (Gradient Descent).
- `lr=0.001` : معدل التعلم (Learning Rate)؛ وهو معامل يحدد حجم الخطوة التي يتخذها المُحسّن أثناء تحديث الأوزان للوصول إلى الاستقرار (Convergence).

```
print("Starting Training...")
for epoch in range(50):
    running_loss = 0.0
    for images, labels in train_loader:
```

الشرح: إجراء 50 دورة تدريبية (Epochs) على كامل البيانات. في كل دورة، يتم سحب حزم من الصور بتصنيفاتها الحقيقية.

```
optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
```

الشرح:

- `optimizer.zero_grad()` : تصفير التدرجات (Gradients) قبل كل حساب جديد لضمان عدم تراكمها من الخطوات السابقة.
- يتم تمرير الصور للنموذج لإنتاج التوقعات، ثم يُحسب مقدار الخطأ `loss`.

```
loss.backward()
optimizer.step()
running_loss += loss.item()
```

الشرح:

- `loss.backward()` : عملية الانتشار العكسي (Backpropagation) لحساب مشتقة الخطأ بالنسبة لكل وزن باستخدام قاعدة السلسلة (Chain Rule).
- `optimizer.step()` : تحديث قيم الأوزان الفعلية لتقليل نسبة الخطأ.
- يتم تجميع قيم الأخطاء لطباعة متوسط الخطأ في نهاية كل دورة.

القسم الخامس: التقييم واختبار الدقة (Evaluation & Accuracy)

```
correct = 0
total = 0
model.eval()
```

الشرح: تحويل النموذج إلى وضع التقييم (`model.eval()`)، مما يؤدي إلى إيقاف العمليات المخصصة للتدريب فقط وتثبيت الأوزان لضمان تقييم دقيق.

```
with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

الشرح:

- `torch.no_grad()` : إيقاف حسابات التدرجات لتوفير الذاكرة وتسريع عملية الاختبار.
- `torch.max` : استخراج الفهرس (Index) صاحب أعلى احتمالية من بين الفئات الثلاث لاعتماده كالتوقع النهائي.
- يتم احتساب عدد الإجابات الصحيحة ومقارنتها بإجمالي الصور.

```
print(f'Final Accuracy: {100 * correct / total:.2f}%')
```

الشرح: حساب وطباعة الدقة النهائية كنسبة مئوية.

القسم السادس: التوقع العملي (Inference)

```
from PIL import Image
import matplotlib.pyplot as plt

def predict_image(image_path):
    img = Image.open(image_path)
    transform = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    img_tensor = transform(img).unsqueeze(0)
```

الشرح:

- إنشاء دالة تستقبل مسار صورة للاختبار الفردي.
- تطبيق نفس معالجات الحجم والتطبيع (بدون Augmentation).
- `unsqueeze(0)`: إضافة بُعد إضافي للمصفوفة لتصبح `[1, 3, 32, 32]` لتتوافق مع متطلبات النموذج الذي يتوقع استقبال حزم (Batches) وليس صوراً منفردة.

```
model.eval()
with torch.no_grad():
    output = model(img_tensor)
    _, predicted = torch.max(output, 1)

class_names = train_dataset.classes
print(f"Prediction: {class_names[predicted.item()]}")
plt.imshow(img)
plt.axis('off')
plt.show()

predict_image('/content/drive/MyDrive/Office_Tools_Project/test/Scissors/
Scissors (5).jpg')
```

الشرح: يُمرر الـ Tensor إلى النموذج لاستخراج النتيجة. يتم تحديد الفئة ذات الاحتمالية الأعلى وطباعة اسمها، ثم عرض الصورة الأصلية. في السطر الأخير، يتم استدعاء الدالة واختبارها على صورة مقص للتأكد من قدرة النموذج على التصنيف الصحيح.

خلاصة المشروع للمناقشة الأكاديمية:

"تم تصميم هذا المشروع بالاعتماد على معمارية الشبكات العصبية التلافيفية (Deep CNN) المبنية من الصفر. تم تفضيل CNN على الشبكات العصبية التقليدية لكونها تحافظ على العلاقات المكانية (Spatial) لتقليل (Weight Sharing) للبكسلات عبر مصفوفات ثنائية الأبعاد، وتوظف آلية مشاركة الأوزان (Features) العبء الحسابي ومنع التجهيز الزائد. تنقسم المعمارية منهجياً إلى مرحلتين: مرحلة استخراج الميزات، وتتألف من 3 طبقات Convolution مدعومة بحشوة (Same Padding) للحفاظ على الحواف، تليها دوال تنشيط ReLU وطبقات Max Pooling للتقليص المكاني. يعقب ذلك مرحلة التصنيف التي تعتمد على التسطيح (Flattening) والطبقات الخطية، لتنتهي بدالة التكلفة CrossEntropyLoss التي توظف Softmax داخلياً لاستخراج الاحتمالات النهائية والتصنيف بدقة عالية."